

MULTIMODAL USER IDENTITY VERIFICATION & PRODUCT RECOMMENDATION SYSTEM

FORMATIVE 2: MULTIMODAL DATA PREPROCESSING ASSIGNMENT

Team Members: Andrew Ogayo, Divine Itu, HonourGod Levison, Keane Gaju

GitHub link: [GitHub](#) | **Contribution Sheet:** [Sheet](#) | **Simulation Video Link:** [Video](#)

ABSTRACT

This report documents the design and implementation of a multimodal user identity verification and product recommendation system. The work spans four stages, each owned by one team member: merging and engineering features from two tabular customer datasets into a leakage-free product recommendation target; collecting and processing facial image data into a facial recognition model; collecting and processing voice audio data into a voiceprint verification model; and integrating all three trained models into a command-line application that enforces a sequential authentication state machine (face check, product inference, voice check, display) before revealing a personalized product recommendation. Three classifiers were trained and compared per task (Logistic Regression, Random Forest, and, for the product model, Gradient Boosting), with Logistic Regression selected as the best-performing candidate in all three cases. The facial recognition model reached 90.0% accuracy ($F1 = 0.900$), the voiceprint verification model reached 97.9% accuracy ($F1 = 0.960$) on a genuine/impostor pairing task, and the product recommendation model reached 50.0% accuracy ($F1 = 0.435$) on a small, 36-customer merged dataset. The full authentication pipeline was verified end-to-end against three required demonstration scenarios: an unauthorized face image, an authorised face paired with an unauthorized voice sample, and a fully authorized transaction.

1. INTRODUCTION AND PROBLEM DEFINITION

The assignment frames a realistic constraint on personalized recommendation systems: a product suggestion should never be shown to a user whose identity has not been verified through more than one independent channel. This project implements that constraint as a four-stage sequential state machine: a user submits a face image, which must match a known team member before anything else happens; the product recommendation model then runs in the background on that user's transaction history, but its output is withheld; the user must then submit a voice sample, which must match the identity established at the face-check stage; only if both biometric checks pass is the recommended product displayed. Any failure at either biometric stage results in an access-denied outcome without ever computing or revealing a recommendation.

Two kinds of data underpin this system. The tabular half, `customer_social_profiles.xlsx` and `customer_transactions.xlsx`, was provided as part of the assignment and is used to engineer features and a leakage-free target for the product recommendation model. The biometric half, facial images and voice recordings, was collected directly from the four team members, since a face-and-voice verification system requires enrolled subjects rather than an anonymized public dataset. Stacking two biometric factors from different modalities (visual and acoustic) is a deliberate design choice: a photograph alone, or a voice recording alone, is a much weaker guarantee of identity than requiring both to agree, which is the same principle underlying multi-factor authentication and multimodal biometric fusion more broadly [6].

The remainder of this report follows the four tasks in the assignment brief. Section 2 covers the tabular data merge and feature engineering behind the product recommendation target. Section 3 covers the image pipeline and facial recognition model. Section 4 covers the audio pipeline and voiceprint verification model. Section 5 covers the product recommendation model itself, the integration of all three models into the command-line application, and the required system demonstration scenarios. Section 6 concludes.

2. DATA MERGE AND PRODUCT RECOMMENDATION FEATURE ENGINEERING

2.1 Understanding the Source Data

The two provided source files describe the same customer population from two different angles: `customer_social_profiles` captures engagement behaviour across social platforms, and `customer_transactions` captures purchase history. Reconciling them required resolving a genuine schema mismatch rather than a simple key match: social profiles identify a customer with a string ID such as "A178" (`customer_id_new`), while transactions use the bare integer 178 (`customer_id_legacy`). The numeric portion of the former was extracted and used as the join key across both tables.

Further data quality issues were identified and handled explicitly rather than silently: social profiles contain multiple rows per customer, one per platform the customer is active on (150 raw rows collapsing to 84 unique customers after aggregation), plus five exact duplicate rows

that were dropped; transactions have missing `customer_rating` values on 10 of 150 rows, imputed using the median rating within the same product category rather than a global median, to avoid distorting category-level patterns. Only 84 customers have a social profile, and 75 have transactions, with 61 customers present in both; after also requiring at least two prior transactions per customer (needed to build a leakage-free target, below), the final merged, model-ready dataset contains 36 customers, which is an expected consequence of intersecting three separate completeness requirements on a modestly sized raw dataset rather than a processing error.

2.2 Feature Engineering and Leakage Control

Social engagement features were aggregated to one row per customer: mean and max engagement score, mean purchase-interest score, a numeric-mapped mean sentiment score, the customer's dominant sentiment and primary platform (each one-hot encoded), platform diversity (count of distinct platforms), and social record count. Transaction features follow a classic Recency-Frequency-Monetary structure: transaction frequency, total and average spend, average rating, favourite category (one-hot encoded), category diversity, recency in days, customer tenure in days, and average purchase cadence in days.

The prediction target, `target_product_category`, is each customer's most recent purchase category, held out from the feature computation entirely: every RFM and behavioural feature above is computed only from transactions strictly earlier than that held-out purchase. This prevents the model from having indirect access to information about the transaction it is being asked to predict, a standard requirement for any target constructed from a customer's own history.

2.3 Result

The final `merged_dataset.csv` contains 36 customers and 30 columns (engineered features plus the target and its integer-encoded form), documented column-by-column in `docs/feature_definitions.md`. Exploratory analysis in `notebooks/Task1_Data_Merge_Feature_Engineering.ipynb` includes summary statistics and variable-type checks, distribution and outlier visualizations (boxplots across the engineered numeric features), and a post-merge correlation heatmap across the full engineered feature set, confirming the RFM features carry distinct, non-redundant signal from the social engagement features rather than duplicating one another.

3. IMAGE DATA COLLECTION AND FACIAL RECOGNITION

3.1 Data Collection

Each of the four team members submitted three facial expressions: neutral, smile, and surprised, stored under `data/raw/images/<member>/` with the consistent naming convention `<member>_<expression>.jpg` (e.g. `andrew_neutral.jpg`). This gives 12 base photographs across the team, one triplet per member, satisfying the assignment's per-member minimum with a uniform, predictable file layout that the preprocessing script can walk without member-specific handling.

3.2 Preprocessing and Visualization

Every photograph is passed through an OpenCV Haar-cascade face detector (`haarcascade_frontalface_default.xml`), based on the boosted-cascade object detection method of Viola and Jones [4], which locates and crops the face region before resizing it to a fixed 128x128 resolution. This step also normalizes away the wide range of source image sizes the team submitted, from roughly 640px compressed messaging-app photos up to 4608px phone camera captures. A face was successfully detected in all 12 submitted photographs. Mean brightness and contrast were found to differ measurably by person (for example, one member's crops average roughly 87 intensity units against another's roughly 137), a difference that itself becomes part of the recognition signal, analogous to how energy differences between speakers later helped separate voices in Section 4.

3.3 Augmentation

Four augmentations were applied per cropped face, doubling the assignment's minimum of two: a 15-degree rotation, a horizontal flip, a grayscale conversion, and a brightness boost (+40 intensity units). Each original photo therefore expands into five stored variants (the original plus four augmentations), visualized side by side in `notebooks/Task2_Image_Pipeline_Divine.ipynb`.

3.4 Feature Extraction

For every original and augmented image, three feature groups are extracted and written to `data/processed/image_features.csv`: an 8x8 downsampled grayscale embedding (64 values, a compact spatial summary of the face crop), a 32-bin pixel-intensity histogram, and the crop's mean and standard deviation intensity. With 4 members x 3 expressions x 5 variants, the resulting feature table has 60 rows.

3.5 Facial Recognition Model

Unlike the voice task (Section 4), where every member had only one recording per phrase, each member here has three distinct base photographs, which is enough to support a genuine multiclass classifier trained directly on the member label rather than requiring a genuine/impostor pairing scheme. Features are standardized and reduced with PCA before classification, since the raw feature space (roughly 100 columns) is wide relative to the 60 available rows. Two candidates were trained and compared: Logistic Regression and Random Forest, using a grouped train/test split that holds out the "surprised" expression (and its

augmented variants) for every member as the test set, training on "neutral" and "smile" instead. This keeps the split class-balanced across all four members while guaranteeing that an augmented copy of a held-out test photo never leaks into training.

Because a plain multiclass classifier always outputs *some* known member, even when shown a stranger's face, `predict_face()` additionally applies a confidence threshold (`UNKNOWN_THRESHOLD = 0.5`) on the top predicted class probability: a low-confidence prediction, or a photo with no detectable face at all, is reported as an unrecognized identity rather than a low-confidence guess. This threshold is what allows the system to correctly reject the unauthorized-attempt image required in Section 5.4.

3.6 Evaluation

Held-out grouped split, 40 train / 20 test rows:

Model	Accuracy	F1 (macro)	Log loss
Logistic Regression (selected)	0.900	0.900	0.242
Random Forest	0.900	0.896	0.557

Logistic Regression was selected on F1. Both models made exactly two mistakes on the 20-row held-out set, and both errors were the same confusable pair in each direction: one member's held-out "surprised" photo was predicted as another specific member, and vice versa, while the remaining two members were classified perfectly. `predict_face(image_path)` -> (is_known_user, predicted_member, confidence) in `scripts/facial_recognition_model.py` is the face gate used by `app/cli_app.py`, and its predicted member also becomes the identity claim passed to the voice-verification stage.

4. AUDIO DATA COLLECTION AND VOICEPRINT VERIFICATION

4.1 Data Collection

Each of the four team members recorded two phrases, "Yes, approve" and "Confirm transaction", stored under `data/raw/audio/<member>/` with the consistent naming convention `<member>_<phrase>.wav`, giving eight clean recordings across the team.

4.2 Preprocessing and Visualization

Recordings are resampled to 16 kHz mono. Waveform and log-frequency spectrogram plots for every original and augmented sample are generated and saved to `data/processed/plots/`, with narrative interpretation in `notebooks/Task3_Audio_Pipeline_HonourGod.ipynb`. Acoustic summary statistics differ measurably by speaker: RMS energy and spectral centroid in particular separate the four voices well (for example, one member's recordings are consistently the loudest while another's sit at a noticeably higher spectral centroid), which is

exactly the kind of per-speaker signature the verification model in Section 4.5 is built to exploit.

4.3 Augmentation

Three augmentations were applied per recording, exceeding the assignment's minimum of two: a pitch shift of +2 semitones, a 1.1x time stretch, and additive Gaussian background noise ($\sigma = 0.01$). The noise augmentation uses a fixed random seed, so the entire pipeline is fully reproducible on rerun rather than producing slightly different features each time it is executed.

4.4 Feature Extraction

For every original and augmented sample, MFCC mean and standard deviation, spectral centroid, spectral rolloff, zero-crossing rate, RMS energy, and clip duration are extracted and written to data/processed/audio_features.csv. With 4 members x 2 phrases x 4 variants (1 original + 3 augmentations), the resulting feature table has 32 rows.

4.5 Voiceprint Verification Model

Voice verification is framed deliberately as a *verification* problem, not a plain classification problem: the question is whether a sample matches the identity it claims to be, not which of several known speakers produced it. This distinction matters concretely here, since with only one recording session per phrase per member, a classifier trained directly on the member label would have no genuine negative examples to learn to reject.

Instead, the model is trained on genuine/impostor pairs. For each member, a voiceprint centroid is computed as the mean of their acoustic features across their original (non-augmented) recordings. Every sample, original or augmented, is then paired with every member's centroid, producing one (sample, claimed_identity) pair labelled genuine when the claim matches the sample's true speaker and impostor otherwise. The model's input is the absolute difference between the sample's features and the claimed centroid, on the premise that small differences indicate a genuine match and large differences indicate an impostor. Train/test splits are grouped by the underlying source recording, so that augmented copies of the same clip never appear on both sides of the split. Logistic Regression and Random Forest were trained and compared; a simple DNN was deliberately not attempted, since eight base recordings per member is far too little data to train a neural network without severe overfitting.

4.6 Evaluation

Held-out grouped split, 80 train / 48 test pairs (128 genuine/impostor pairs total):

Model	Accuracy	F1	Log loss
Logistic Regression (selected)	0.979	0.960	0.182

Random Forest	0.979	0.960	0.117
---------------	-------	-------	-------

Accuracy and F1 tie exactly between the two candidates, with both producing the same confusion matrix on the held-out set; the selection rule keeps Logistic Regression as the simpler model when F1 is tied. `verify_voice(claimed_member, audio_features) -> (is_verified, confidence)` in `scripts/voice_verification_model.py` is the voice gate used by `app/cli_app.py`. Spot checks against the trained model confirm the intended behaviour directly: a sample claimed under its true speaker's identity is verified with moderate-to-high confidence, while the identical sample claimed under a different team member's identity is rejected with a confidence near zero.

5. PRODUCT RECOMMENDATION MODEL, SYSTEM INTEGRATION, AND DEMONSTRATION

5.1 Product Recommendation Model

`scripts/product_recommendation_model.py` trains a multiclass classifier predicting `target_product_category` from the numeric feature columns of `merged_dataset.csv` produced in Section 2, excluding identifier and label-leakage columns. Three candidates were trained and compared: Logistic Regression, Random Forest, and Gradient Boosting (used as a dependency-light stand-in for XGBoost).

Held-out split, 28 train / 8 test rows, 5 target classes:

Model	Accuracy	F1 (weighted)	Log loss
Logistic Regression (selected)	0.500	0.435	2.375
Gradient Boosting	0.375	0.267	5.152
Random Forest	0.250	0.125	1.826

Logistic Regression was selected on F1. Its accuracy is markedly weaker than the facial or voice models, which is an honest reflection of the underlying data rather than a modeling defect: the fully merged, leakage-free dataset has only 36 customers spread across 5 target categories, leaving an 8-row test set in which a single misclassification already costs 12.5 percentage points of accuracy. The biometric stages of the system are unaffected by this limitation and continue to block unauthorized access regardless of how confident the eventual product recommendation is.

5.2 System Integration

app/cli_app.py implements the four-stage state machine described in Section 1 as a linear sequence with an AccessDenied exception used to short-circuit the flow on failure:

1. **Face check:** predict_face() must recognize a known team member, or the transaction is denied immediately.
2. **Product inference:** predict_product() runs against the identified customer's row in merged_dataset.csv, but the result is held in memory and not displayed.
3. **Voice check:** verify_voice() must confirm the claimed identity established at Stage 1, or the transaction is denied.
4. **Display:** only once both biometric checks have passed is the recommended product, computed silently at Stage 2, printed to the user.

Per-model evaluation reports are written to data/processed/*_model.json by each model's own training script, following the same pattern in all three cases: train several candidate algorithms, evaluate on a held-out split, keep the best by F1, and persist both the trained pipeline and a JSON metrics report. The combined results across all three models:

Model	Selected algorithm	Accuracy	F1	Log loss
Facial recognition	Logistic Regression	0.900	0.900	0.242
Voice verification	Logistic Regression	0.979	0.960	0.182
Product recommendation	Logistic Regression	0.500	0.435	2.375

5.3 System Demonstration

app/simulate_scenarios.py runs the three scenarios required by the assignment brief against the live, trained models, and prints a pass/fail summary:

Scenario	Expected outcome	Result
Unauthorized face image	Denied at Stage 1 (face)	Denied at Stage 1
Authorized face, unauthorized voice sample	Denied at Stage 3 (voice)	Denied at Stage 3
Authorized face and voice,	Granted, product displayed	Granted at Stage 4

valid customer	at Stage 4	
----------------	------------	--

All three scenarios produced their expected outcome. This confirms two things beyond the individual model metrics in Section 5.2: first, that the end-to-end pipeline runs correctly against live model artifacts rather than only in isolated unit tests; and second, that stacking two independent biometric checks does what it is meant to do, an attacker who somehow passes the face check is still caught at the voice check, which is the entire justification for using two modalities instead of one [6]. An interactive single-transaction mode is also available via `python -m app.cli_app` for manual demonstration.

6. REPOSITORY AND CONCLUSION

This project implemented a complete multimodal identity verification and recommendation pipeline: a leakage-free tabular feature engineering process behind a product recommendation target; an image collection, augmentation, and facial recognition pipeline; an audio collection, augmentation, and voiceprint verification pipeline built around a genuine/impostor pairing strategy; and a command-line application that integrates all three trained models into the sequential authentication flow specified by the assignment. All three models were trained, evaluated with accuracy, F1, and log loss, and persisted as reusable artefacts, and the full system was verified end-to-end against all three required demonstration scenarios.

The repository contains the two provided source datasets and the team-collected image and audio data under `data/raw/`; the merged and engineered `merged_dataset.csv`, `image_features.csv`, and `audio_features.csv` under `data/processed/`, alongside every trained model's serialized pipeline and JSON evaluation report; the reusable pipeline scripts for all three models under `scripts/`; the four per-task Jupyter notebooks under `notebooks/`; and the command-line application and scenario simulation under `app/`. Full setup and run instructions are documented in the repository's top-level `README.md`.

TEAM CONTRIBUTIONS

Member	Contribution
Andrew	Data merge, cleaning, and feature engineering for the product recommendation target (<code>merged_dataset.csv</code> , <code>scripts/data_preprocessing.py</code> , EDA notebook)
Divine	Image collection, preprocessing, augmentation, and the facial recognition model
HonourGod	Audio collection, preprocessing, augmentation, and the voiceprint verification model
Gaju	Product recommendation model, CLI system integration, system simulation, report assembly

REFERENCES

- [1] Course-provided datasets, customer_social_profiles.xlsx and customer_transactions.xlsx, ALU Formative 2: Multimodal Data Preprocessing Assignment.
- [2] D. R. Cox, "The regression analysis of binary sequences," *J. R. Stat. Soc. Ser. B*, vol. 20, no. 2, pp. 215-242, 1958.
- [3] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5-32, Oct. 2001.
- [4] P. Viola and M. Jones, "Rapid object detection using a boosted cascade of simple features," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2001, pp. I-511-I-518.
- [5] S. B. Davis and P. Mermelstein, "Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences," *IEEE Trans. Acoust., Speech, Signal Process.*, vol. 28, no. 4, pp. 357-366, Aug. 1980.
- [6] A. K. Jain, A. Ross, and S. Prabhakar, "An introduction to biometric recognition," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 14, no. 1, pp. 4-20, Jan. 2004.
- [7] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825-2830, 2011. [Online]. Available: <https://scikit-learn.org/>
- [8] B. McFee et al., "librosa: Audio and music signal analysis in Python," in *Proc. 14th Python in Science Conf. (SciPy)*, 2015, pp. 18-24. [Online]. Available: <https://librosa.org/>
- [9] G. Bradski, "The OpenCV Library," *Dr. Dobb's J. Softw. Tools*, 2000. [Online]. Available: <https://opencv.org/>